



EfficientServe: QoS-Guaranteed LLM Serving for Mixed Downstream Workloads

Jiahao Huang^{1,2}, Tao Wang^{*2}, Yuewen Wu², Zhen Tang², Heng Wu², Wenbo Zhang^{2,3}

¹University of Chinese Academy of Sciences, Beijing, China

²Institute of Software, Chinese Academy of Sciences, Beijing, China

³Nanjing Institute of Software Technology, Nanjing, China

{huangjiahao23, wangtao08, wuyuewen, tangzhen12, wuheng, zhangwenbo}@otcaix.iscas.ac.cn

Abstract—Large Language Models (LLMs) have demonstrated remarkable capabilities, leading to a surge in real-world application deployments. However, serving LLMs in production environments faces unprecedented challenges due to high concurrency and diverse query types. Real-world mixed workloads typically comprise both latency-sensitive interactive tasks and throughput-oriented computational tasks. Existing mainstream LLM inference systems predominantly rely on simple First-Come-First-Served scheduling and rigid KV cache management. These approaches often suffer from severe memory bottlenecks and Head-of-Line blocking, making it difficult to guarantee Quality of Service (QoS) and resulting in low Service Level Objective (SLO) attainment rates. To overcome these limitations, we propose EfficientServe, a high-performance LLM inference serving system tailored for multi-scenario mixed workloads. First, we introduce an embedding-based iterative length prediction method utilizing the intermediate layer features of LLMs. Second, we design an adaptive dynamic cache allocation mechanism that incorporates a high-compression H Cache to alleviate memory pressure during traffic bursts. Finally, we develop a QoS-aware scheduling strategy that intelligently interleaves tasks based on predicted lengths and cache states. Extensive experiments show that EfficientServe enhances load capacity by $1.9\times$ to $2.6\times$ compared to state-of-the-art inference systems while robustly maintaining a 90% SLO attainment rate. The source code for EfficientServe is available at <https://github.com/chiahao-huang/EfficientServe>.

Index Terms—LLM Serving, Mixed Workloads, Task Scheduling, Cache Management

I. INTRODUCTION

With the emergence of large language models including ChatGPT [1], Llama [2], Qwen [3], DeepSeek [4], Gemini [5], and Kimi [6], LLMs have fundamentally revolutionized artificial intelligence, shifting the field from discriminative tasks toward powerful generative applications. Today, LLM services are ubiquitous, powering everything from real-time conversational agents to offline data processing and complex reasoning tasks. As these applications scale, the demand for efficient, high-throughput inference serving systems has become a critical bottleneck.

The autoregressive nature of LLM inference intrinsically divides the generation process into a prefill phase and a decode phase. As the context length and batch size increase, the memory required to store the KV Cache grows linearly, creating

a severe memory wall. Furthermore, production environments are characterized by mixed workloads, where latency-sensitive requests (prioritizing Time-to-First-Token, TTFT) coexist with throughput-oriented requests. Existing systems predominantly employ First-Come-First-Served (FCFS) scheduling. In a mixed workload scenario, this naive approach leads to Head-of-Line blocking: long, compute-heavy queries monopolize GPU resources, starving shorter, latency-sensitive requests and severely degrading the overall SLO attainment.

To address the endogenous conflicts between memory constraints and scheduling efficiency, we design and implement EfficientServe. Our primary contributions are summarized as follows:

- We propose a lightweight task length prediction architecture that utilizes intermediate layer features and Bayesian smoothing to dynamically estimate generation lengths with negligible overhead.
- We design an adaptive dynamic cache allocation mechanism, introducing “H Cache” to temporarily compress and store hidden states, effectively absorbing sudden spikes in memory demand.
- We develop a QoS-aware scheduling algorithm that leverages length predictions to schedule tasks intelligently, minimizing HoL blocking and maximizing Quality of Service.

II. BACKGROUND AND MOTIVATION

A. Autoregressive Inference and KV Cache Memory Wall

Large Language Models (LLMs) based on the Transformer architecture generate text using an autoregressive paradigm, which essentially consists of two stages: the *prefill* phase and the *decode* phase. During the prefill phase, the model processes the entire input prompt in parallel to compute the initial key (K) and value (V) vectors. In the subsequent decode phase, the model generates tokens one by one sequentially. To avoid redundant computations of previously generated tokens, inference systems cache the K and V tensors of all past tokens in the GPU memory, widely known as the KV Cache.

Although the KV Cache significantly accelerates the decode phase, it introduces a severe memory bottleneck. The memory footprint of the KV Cache grows linearly with both the

*Corresponding author: wangtao@otcaix.iscas.ac.cn

sequence length and the batch size. In a typical deployment scenario, the GPU’s High Bandwidth Memory is rapidly consumed by the dynamically growing KV Caches rather than the static model weights. Once the HBM is exhausted, the system is forced to preempt tasks or trigger Out-Of-Memory errors, severely capping the maximum achievable concurrency and throughput of the serving system.

B. Heterogeneity in Mixed Application Scenarios

In production environments, LLM serving systems rarely handle a single type of request. Instead, they face *mixed workloads* characterized by highly diverse application scenarios and resource demands. These workloads typically encompass two main categories:

- **Latency-Sensitive Tasks:** Interactive applications such as chatbots, coding assistants, and real-time QA systems require immediate responses. They demand strict Service Level Objectives (SLOs), specifically a very low Time-to-First-Token (TTFT) and a smooth Time-Between-Tokens (TBT), to ensure a seamless user experience.
- **Throughput-Oriented Tasks:** Background processing applications, such as offline document summarization, large-scale data translation, and log analysis, are compute-intensive. They typically process massive context windows and generate exceptionally long sequences. These tasks have a higher tolerance for initial latency but prioritize overall system throughput and Time-to-Last-Token (TTLT).

This inherent heterogeneity creates conflicting resource demands. Latency-sensitive tasks require immediate but brief access to computational resources, whereas throughput-oriented tasks tend to monopolize both compute cores and memory capacity for extended periods.

C. Head-of-Line Blocking and Scheduling Conflicts

The fundamental disparity in mixed workloads exposes the critical flaws of traditional scheduling algorithms. Existing mainstream LLM inference engines, primarily rely on simple First-Come-First-Served or static batching strategies. Under FCFS, when a throughput-oriented task with a long generation length enters the compute stream, it occupies a large portion of the GPU resources.

Consequently, subsequent latency-sensitive requests are forced to wait in the queue until the long task completes or is statically swapped out. This phenomenon is known as Head-of-Line blocking. The inability of the scheduler to preemptively distinguish between task urgencies or foresee the remaining generation length leads to massive TTFT violations for interactive queries. Furthermore, traditional systems use rigid memory allocation, forcing eviction and expensive re-computation when bursts occur. Therefore, addressing the HoL blocking problem through a predictive, Quality-of-Service aware scheduling mechanism, coupled with dynamic memory elasticity, is the primary motivation behind the design of EfficientServe.

III. EFFICIENTSERVE DESIGN

A. System Architecture Overview

To fundamentally resolve the inherent conflicts between computational throughput and memory constraints in mixed-workload scenarios, EfficientServe adopts a highly decoupled, state-aware system architecture as shown in Fig. 1. The pipeline is designed to maximize resource liquidity and execution elasticity. The system primarily consists of three cooperative core components: the Embedding-based Iterative Length Predictor, the Adaptive Dynamic Cache Manager, and the QoS-aware Scheduler.

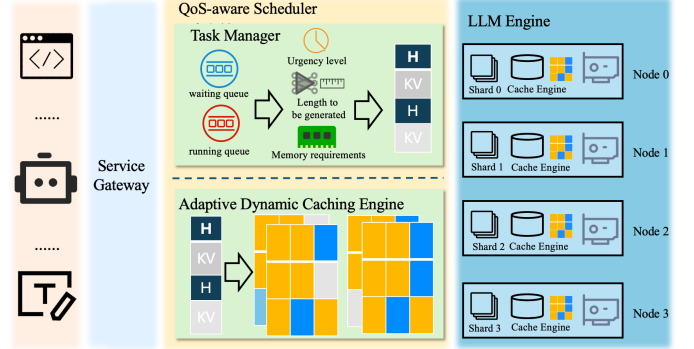


Fig. 1: An overview of EfficientServe.

When a new user request arrives, it is initially intercepted by the Length Predictor, which performs a lightweight forward pass to establish a probabilistic estimation of the expected generation length. Subsequently, the request enters the global scheduling queue. During each iteration of the autoregressive decoding cycle, the QoS-aware Scheduler comprehensively evaluates the multi-dimensional priority value of all active and queued requests. Simultaneously, the Adaptive Cache Manager monitors the physical usage of the unified GPU memory pool in real time, dynamically converting memory blocks between standard KV Cache and the highly compressed H Cache. This coordinated, closed-loop pipeline ensures that latency-sensitive interactive queries achieve sub-second TTFT, while compute-intensive background tasks progress steadily without triggering system-wide out-of-memory failures.

B. Embedding-Based Iterative Length Prediction

Accurate and dynamic sequence length prediction is the prerequisite for preemptive scheduling. Conventional methods either rely on naive static estimation [7]—which leads to massive memory fragmentation—or employ heavy external models [8]–[11] that incur unacceptable latency overhead. EfficientServe proposes an internal feature-extraction approach.

Feature Extraction: The autoregressive nature of LLMs means that early output tokens already encapsulate the core semantic intent of the entire sequence. Empirical studies within our system reveal that hidden state vectors derived from the intermediate layers of the LLM (specifically near layer $L/2$) exhibit the strongest correlation with the final output length. These intermediate representations strike an optimal balance,

capturing high-level syntactic structures without being overly biased toward immediate next-token prediction.

Model Architecture and Bayesian Smoothing: We deploy a lightweight Multi-Layer Perceptron (MLP) over these intermediate embeddings. The hidden states are aggregated via average pooling, followed by a two-layer MLP that outputs a probability distribution across K predefined, non-uniform logarithmic length buckets. To mitigate sequence-level jitter during generation, EfficientServe employs a Bayesian iterative smoothing algorithm. Let π_{t-1} be the prior probability distribution from the previous decoding step, and E_t be the current observation from the MLP. The smoothed posterior prediction $\hat{\pi}_t$ is calculated as:

$$\hat{\pi}_t(y = k) = \frac{\pi_{t-1}(y = k) \cdot E_t(y = k)}{\sum_{j=1}^K \pi_{t-1}(y = j) \cdot E_t(y = j)} \quad (1)$$

This mechanism continuously calibrates the length estimation, ensuring that the predicted remaining length monotonically decreases in a stable manner, providing highly reliable signals for the scheduler.

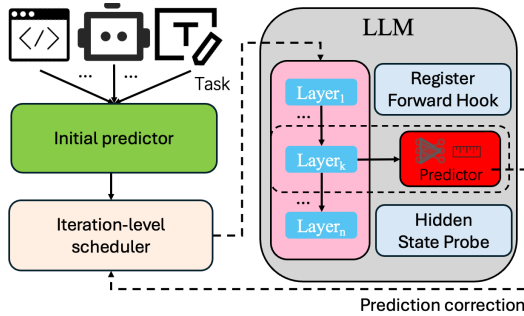


Fig. 2: Embedding-based iterative length prediction architecture.

C. Adaptive Dynamic Cache Allocation Mechanism

The fundamental bottleneck of LLM inference is the memory wall. The total KV Cache size M_{KV} scales linearly with sequence length and batch size, calculated as:

$$M_{KV} = 2 \times S \times L \times h_{model} \times P \quad (2)$$

where S is the sequence length, L represents the number of Transformer layers, h_{model} is the hidden dimension size, and P denotes the data precision bytes.

To overcome this, we introduce the H Cache (Hidden State Cache) mechanism backed by a unified global memory pool. While traditional KV Cache stores the full Key and Value matrices across all attention heads, H Cache strictly stores the intermediate hidden state vectors. For standard Multi-Head Attention (MHA) architectures, H Cache achieves a theoretical compression ratio of 0.5, effectively halving the memory footprint.

EfficientServe implements a paged memory manager where physical GPU memory is divided into fixed-size blocks dynamically mapped as either KV_BLOCK or H_BLOCK. Under normal load, standard KV Cache is utilized to guarantee

maximum decoding speed. However, when memory utilization exceeds a critical high-water mark during traffic bursts, the Cache Manager adaptively compresses the KV Caches of low-priority background tasks into H Cache. When these preempted tasks are rescheduled, the system performs a partial synchronous recomputation (executing only the top half of the transformer layers) to transparently recover the full KV Cache from the H Cache. This effectively trades a manageable fraction of computational power for critical memory space, avoiding disastrous request evictions.

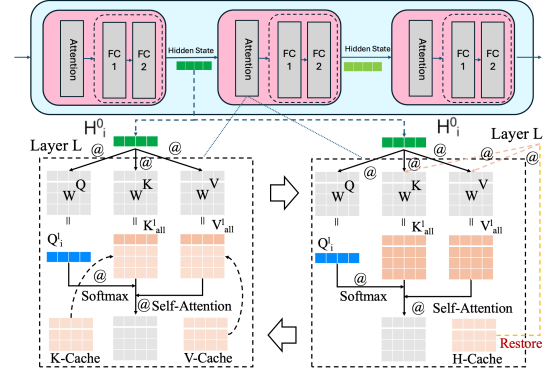


Fig. 3: Mathematical Principles of Adaptive Dynamic Cache Allocation Mechanism.

To better illustrate the implementation principles, this paper presents an example in Fig. 4, which demonstrates how the KV cache and H cache of two different requests are co-managed within a unified memory pool. In this example, the unified memory pool consists of 16 cache blocks (vertically) with a block size of 4 (horizontally), meaning each block can accommodate K/V/Hcaches for 4 token positions. Request A in the figure is allocated KV cache and has a total of 11 tokens (7 prompt tokens + 4 output tokens). Its K cache occupies blocks 2, 4, and 6 in the unified memory pool, while its V cache occupies blocks 10, 11, and 14. In contrast, Request B has 12 tokens (8 prompt tokens + 4 output tokens) and is allocated H cache, placing its hidden cache in blocks 0, 7, and 9. In summary, the different types of caches for these two requests are partitioned into multiple cache blocks. The hybrid cache allocator selects specific cache blocks on-demand for target requests by scanning the available unused cache blocks in the unified memory pool.

D. QoS-Aware Task Scheduling

Mixed workloads suffer from endogenous conflicts. FCFS scheduling inevitably leads to Head-of-Line blocking. EfficientServe addresses this by formulating a scheduling value function that holistically balances task urgency, remaining workload, and memory transition costs. The priority value V_r^i for a request r at iteration i is defined as:

$$V_r^i = \frac{T_r^i}{\lambda \cdot L_r^i} - \mu \cdot \varphi_r^i \cdot (|W^i| + |R^i|) \cdot m_r^i \quad (3)$$

where T_r^i acts as a linear aging function representing accumulated waiting time to prevent starvation. L_r^i denotes the predicted remaining length, favoring Shortest Remaining Processing Time (SRPT). The parameter $\varphi_r^i \in \{0, 1\}$ indicates whether the request currently resides in H Cache mode. The penalty term quantifies the systemic overhead of H Cache recomputation affecting the entire batch.

Based on this value function, the ultimate goal of the scheduler at iteration i is to select an optimal combination of request states $\{(\theta_r^i, \varphi_r^i)\}$ from the candidate pool C^i to maximize the total scheduling utility of the system, while strictly adhering to physical memory constraints. This is mathematically modeled as a combinatorial optimization problem:

$$\text{Maximize } J = \sum_{r \in C^i} V_r^i(\theta_r^i, \varphi_r^i) \cdot \theta_r^i \quad (4)$$

$$\text{s.t. } \sum_{r \in C^i} \text{MemCost}(r) \cdot \theta_r^i \leq M_{\text{available}}^i \quad (5)$$

where the memory cost function $\text{MemCost}(r)$ precisely reflects the physical characteristics of the H Cache:

$$\text{MemCost}(r) = \left(1 - \frac{\varphi_r^i}{2}\right) \cdot m_r^i \quad (6)$$

When $\varphi_r^i = 1$, the memory cost coefficient is 0.5 (saving half the memory); when $\varphi_r^i = 0$, it is 1.0 (indicating full KV Cache occupation). Here, let $\bar{m}_r^i$ denote the currently allocated physical memory blocks for request r at iteration i . $M_{\text{available}}^i$ is the total number of free physical memory blocks at the current iteration. **Complexity Analysis:** This formulation is a variant of the multi-dimensional knapsack problem with state selection. Each request acts as an item with two potential volume/value states (H Cache or KV Cache). The objective is to pack the combination yielding the maximum total utility. Since this problem is fundamentally NP-Hard, employing exact solvers (e.g., integer linear programming) or dynamic programming within the millisecond-level scheduling cycle of an LLM inference engine is entirely infeasible. Therefore, a heuristic algorithm with a good approximation ratio and extremely low time complexity is required.

To operationalize this and efficiently solve the optimization problem, we design a marginal gain-based greedy scheduling algorithm, detailed in Algorithm 1. Instead of naively ranking requests by their absolute value V_r^i , the scheduler evaluates the *marginal gain* (ω) of memory allocation actions. The marginal gain is defined as the potential increase in the scheduling value divided by the required memory footprint ($\omega = \Delta V / \Delta m$).

E. QoS-Aware Inference Task Scheduling

Specifically, the decision space is divided into state transitions. For newly admitted or completely suspended requests, the scheduler calculates the basic admission gain of allocating an H Cache block. For requests already running in H Cache mode, it evaluates the service upgrade gain of transitioning to a full KV Cache, which eliminates the recomputation penalty. At each scheduling tick, the system constructs a pool of these

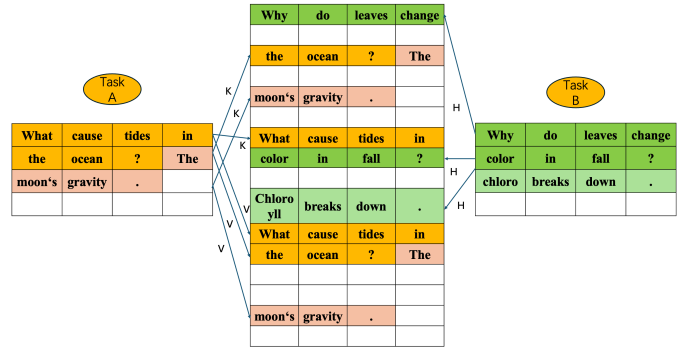


Fig. 4: Mathematical Principles of Adaptive Dynamic Cache Allocation Mechanism.

Algorithm 1 Marginal-Gain QoS-Aware Greedy Scheduling

Require: Candidate set $C^i = W^i \cup R^i$, Available memory M_{avail}^i , Predictor P_θ
Ensure: Decision set $\Theta = \{(\theta_r^i, \varphi_r^i) \mid r \in C^i\}$
1: $A \leftarrow \emptyset$; $\Theta \leftarrow \{(0, 0) \mid \forall r \in C^i\}$ // Init actions & suspend all
2: **for each** request $r \in C^i$ **do**
3: $L_r^i \leftarrow P_\theta(r.\text{emb})$; Compute V_r^i
4: **if** $\bar{m}_r^i = 0$ **then**
5: $\omega \leftarrow V_r^i(\varphi = 1) / (0.5 \cdot m_r^i)$
6: $\text{type}, \Delta m \leftarrow$ **if** $\omega < 0$ **then** (KV, m_r^i) **else** (H, $0.5 \cdot m_r^i$)
7: $A \leftarrow A \cup \{(r, \text{type}, \Delta m, \omega)\}$
8: **else if** $\bar{m}_r^i = 0.5 \cdot m_r^i$ **then**
9: $\omega \leftarrow [V_r^i(\varphi = 0) - V_r^i(\varphi = 1)] / (0.5 \cdot m_r^i)$
10: $A \leftarrow A \cup \{(r, \text{KV}, 0.5 \cdot m_r^i, \omega)\}$
11: **else if** $\bar{m}_r^i = m_r^i$ **then**
12: $A \leftarrow A \cup \{(r, \text{KV}, 0, 0)\}$
13: **end if**
14: **end for**
15: **Sort** A by marginal gain ω in descending order
16: **for each** action $a \in A$ **do**
17: **if** $a.\Delta m \leq M_{\text{avail}}^i$ **then**
18: Apply action a to update $(\theta_r^i, \varphi_r^i)$ in Θ
19: $M_{\text{avail}}^i \leftarrow M_{\text{avail}}^i - a.\Delta m$
20: **end if**
21: **end for**
22: **return** Θ

atomic state-transition actions, ranks them strictly by their marginal gain ω in descending order, and greedily applies them until $M_{\text{available}}^i$ is exhausted. This strategy ensures that limited memory blocks are continuously allocated to actions yielding the highest systemic QoS improvement.

IV. EVALUATION

A. Experimental Setup

Hardware and Software: We implement EfficientServe on a high-performance computing cluster. Single-GPU experiments are conducted on an NVIDIA A100 (40GB) GPU, while

multi-GPU scalability tests utilize a dual-node NVIDIA A100 setup connected via NVLink [12]. The software stack includes Ubuntu 22.04, CUDA 12.1, and PyTorch 2.3.0.

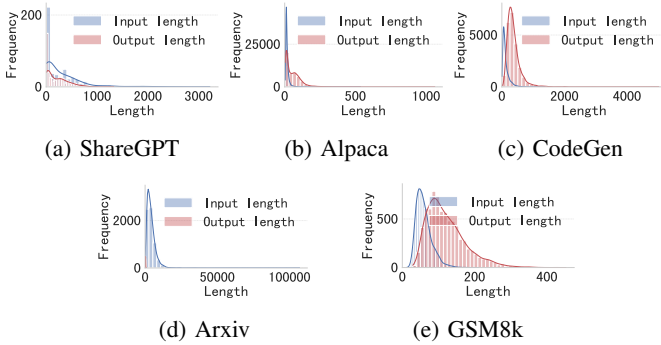


Fig. 5: Input and output sequence length distributions across different datasets.

Models and Baselines: We evaluate the system using Qwen-14B [13] for single-GPU tests and Qwen2.5-32B-Instruct [14] for tensor parallelism (TP=2) tests. We compare EfficientServe against three state-of-the-art baselines: vLLM (representing optimal memory paging) [15], Sarathi-Serve (representing chunked-prefill scheduling) [16], and DeepSpeed-FastGen (representing dynamic split-fuse scheduling) [17].

Workloads and Metrics: We synthesize workloads using five diverse datasets: ShareGPT [18], Alpaca [19], CodeGen [20], Arxiv-Summarization [21], and GSM8k [22]. To illustrate the inherent heterogeneity of these tasks, Fig.5 presents the varying distributions of input and output sequence lengths across the selected datasets. A “Mixed” dataset is constructed by randomly sampling from these to simulate heterogeneous production environments. Furthermore, to evaluate real-world robustness, we replay traffic traces from the BurstGPT [23] dataset. The primary evaluation metric is the SLO attainment rate, defined by strict Time-to-First-Token (TTFT) and Time-Between-Tokens (TBT) constraints.

B. Length Prediction Accuracy and Overhead

We first evaluate the standalone performance of our embedding-based iterative length predictor (EmPre). We compare it against a traditional external BERT-based predictor and an ablated version without Bayesian smoothing.

As shown in Fig. 6, EmPre achieves a significantly lower Mean Absolute Error (MAE) across all datasets compared to the BERT-based method. EmPre outperforms its ablated version without Bayesian smoothing, demonstrating that the smoothing mechanism successfully enhances prediction accuracy. Meanwhile, we observe that the ratio of the predicted value to the ground truth decreases monotonically as the generation process progresses, as shown in Figure 8. This indicates that iterative prediction is very necessary because it can provide us with increasingly accurate predictions. Crucially, the EmPre MLP contains only 6.3M parameters. The results in Fig. 8 indicate that the inference overhead is between 0.6

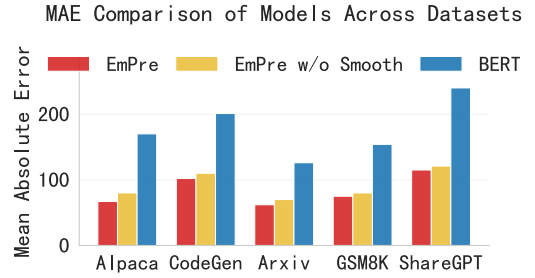


Fig. 6: Comparison of length prediction MAE and dynamic error convergence across different models.

ms and 1.2 ms. This differs by several orders of magnitude from the time taken for a single LLM forward pass, ensuring it hardly affects the main model’s inference speed.

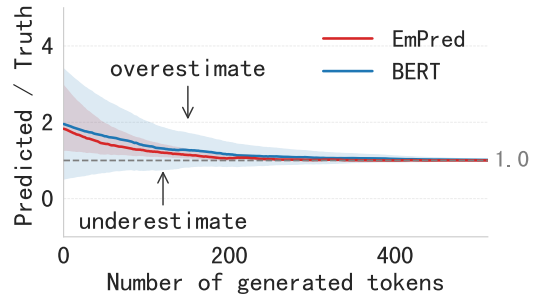


Fig. 7: The ratio of the predicted value to the ground truth changes with the progress of generation.

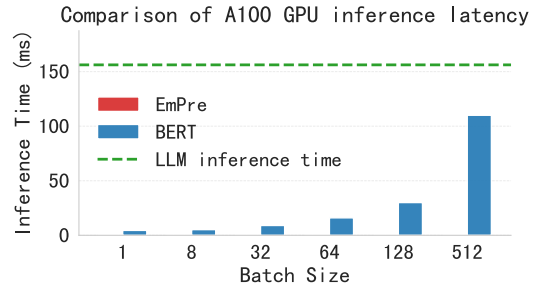


Fig. 8: Comparison between the time of prediction models and the single iteration time of the inference model.

C. End-to-End System Performance

We evaluate the systems under the mixed workload using a Poisson arrival process. Traditional systems suffer from severe Head-of-Line blocking when long-sequence throughput tasks monopolize resources, starving interactive tasks.

Fig. 9(a) demonstrates that EfficientServe effectively mitigates this issue. Under a strict 90% SLO attainment target, EfficientServe supports a maximum request arrival rate that is 2.6x that of vLLM, 2.1x of Sarathi-Serve, and 1.9x of FastGen. When the system operates under extreme overload conditions

(e.g., targeting a 60% SLO), EfficientServe’s throughput advantage expands up to 4.8x over vLLM. This massive gain is attributed to the H Cache mechanism, which virtually doubles the available context capacity, allowing the QoS-aware scheduler to schedule tasks more aggressively. Furthermore, considering the particularity of mixed loads, we also conducted separate experimental analyses on single loads, the results are also shown in Figs. 9 (b)–9(f).

D. Robustness under Burst Traffic

Real-world traffic rarely follows a smooth Poisson distribution. To test system resilience, we replay traces from BurstGPT, characterized by long-tail distributions and sudden concurrency spikes. For the convenience of the experiment, we sampled 108 seconds of data from the high-burst interval of the open-source BurstGPT dataset as the experimental load. Its burst fluctuations are shown in Fig 10. Meanwhile, we also analyzed the input and output distribution of the sampled load, and the results are shown in Fig 11.

Under the BurstGPT trace, baseline systems experience catastrophic cascading violations during traffic surges due to hard OOM limits. In contrast, EfficientServe dynamically compresses low-priority KV Caches into H Caches, absorbing the traffic shock. As depicted in Fig. 12, EfficientServe surpasses the SLO attainment of vLLM, Sarathi-Serve, and FastGen by 22%, 17%, and 15%, respectively, proving its readiness for industrial deployment.

E. Ablation Study

To rigorously validate the necessity and effectiveness of our core designs, we conduct ablation experiments by isolating the Adaptive Dynamic Cache Allocation mechanism and the QoS-aware Scheduling strategy.

Impact of Adaptive Cache Allocation: We first evaluate the system without the H Cache mechanism (denoted as EfficientServe-w/o-H). Under high-burst scenarios (e.g., Coefficient of Variation = 5), the absence of memory elasticity leads to frequent Out-Of-Memory triggers. Consequently, the system is forced to systematically evict and recompute tasks, resulting in a significant drop in SLO attainment (by over 5.5%) and a sharp increase in tail latency.

Impact of QoS-Aware Scheduling: Next, we replace our marginal-gain QoS scheduler with a standard FCFS strategy (denoted as EfficientServe-FCFS) while retaining the H Cache pool. As illustrated in Fig. 14, without the predictive interleaving of tasks, long throughput-oriented requests severely block interactive queries. This Head-of-Line (HoL) blocking causes TTFT violations to skyrocket, demonstrating that intelligent task scheduling is indispensable for mixed workloads.

F. Scalability Analysis

To demonstrate that EfficientServe is a robust system-level solution capable of integrating with other orthogonal optimizations and scaling across hardware architectures, we conduct extensive scalability experiments.

Integration with Chunked-Prefill: EfficientServe’s memory management and scheduling logic are inherently compatible with compute-focused optimizations. We integrate the Chunked-Prefill technique (EfficientServe-CP), which splits the prefill phase of long sequences to avoid GPU compute stalls. As shown in Fig. 15, EfficientServe-CP further boosts the maximum request throughput by 2.3x compared to the native Sarathi-Serve under a strict 90% SLO target.

Distributed Parallelism: Finally, we evaluate the system’s scalability in a multi-GPU environment. We deploy the Qwen2.5-32B model using Tensor Parallelism (TP=2) across dual NVIDIA A100 GPUs. The results in Fig. 16 confirm that our dynamic cache pool and scheduling algorithms introduce negligible synchronization overhead, maintaining near-linear scaling efficiency and proving EfficientServe’s readiness for large-scale distributed deployments.

G. System Overhead Analysis

Introducing complex prediction models and scheduling algorithms inevitably incurs additional computational overhead. We quantify this overhead through micro-benchmarks, as shown in Table. I.

TABLE I: Overhead Analysis of System Algorithms (Unit: ms)

Number of Tasks	32	64	128	512	1024
Prediction	0.6	0.6	0.8	1.2	2.5
Greedy Selection	0.2	0.3	0.6	2.8	6.6
Total Overhead	0.8	0.9	1.4	4.0	9.1

The total overhead during each inference iteration consists of the length prediction and the greedy selection process. The EmPre model, based on a lightweight MLP, takes only 0.6 to 2.5 ms. The greedy selection takes 0.6 ms for 128 tasks and remains under 6.6 ms even for extreme high-concurrency scenarios ($N = 1024$). Consequently, the total scheduling overhead ranges from merely 0.8 ms to 9.1 ms. Considering that a single LLM inference iteration typically requires approximately 100 ms, this overhead is practically negligible and has a very limited impact on the end-to-end latency.

V. RELATED WORK

A. KV Cache Optimization

To alleviate the massive memory footprint of the KV Cache, various compression and eviction techniques have been actively explored. Token-dropping methods, such as H2O [24] and Scissorhands [25], dynamically evict “less important” tokens based on attention scores to maintain a bounded cache budget. InfiniGen [26], on the other hand, proposes an innovative KV cache management framework that improves the efficiency of long text generation by prefetching key data. While effective in reducing memory usage, these approximate methods can inadvertently degrade the model’s reasoning

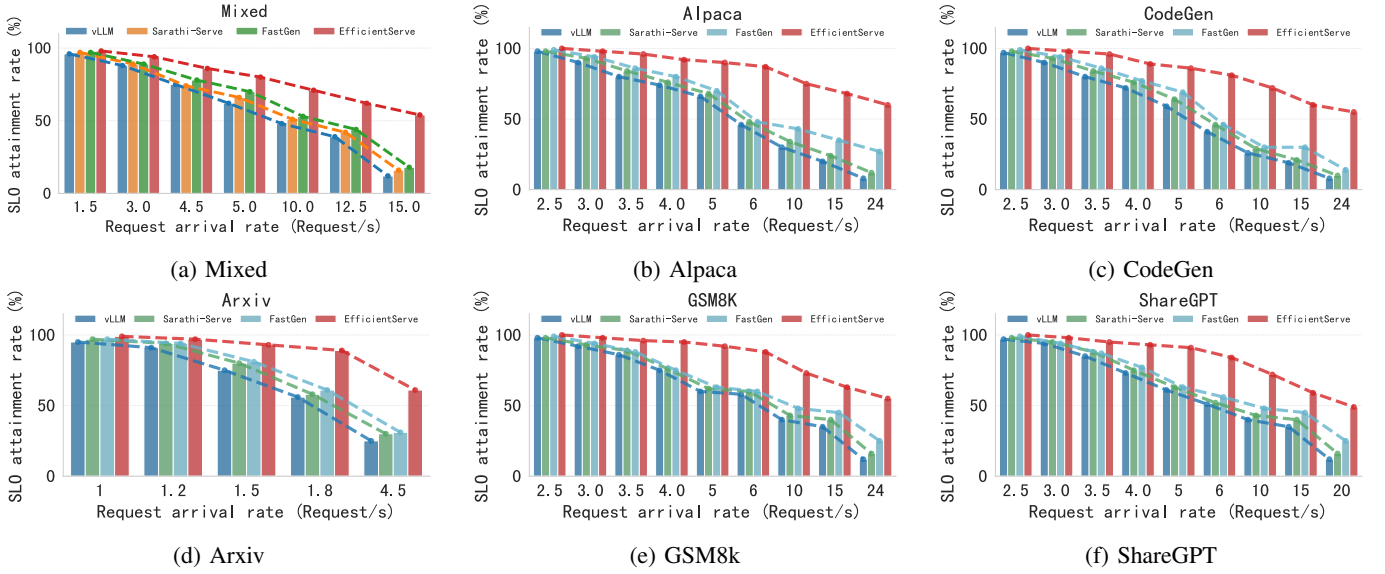


Fig. 9: Comparison of SLO achievement rates between the ou rs and baseline under different single data loads

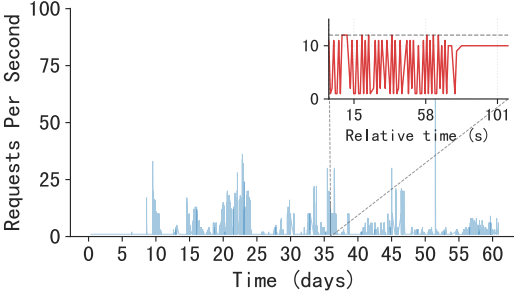


Fig. 10: Schematic diagram of the sampled data load fluctuations from BurstGPT.

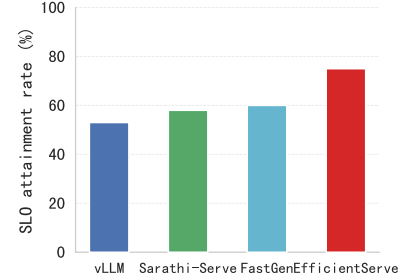


Fig. 12: System SLO attainment rates under real-world BurstGPT traffic traces.

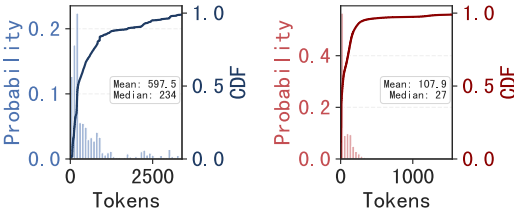


Fig. 11: The distribution of input and output lengths of sampled loads in BurstGPT

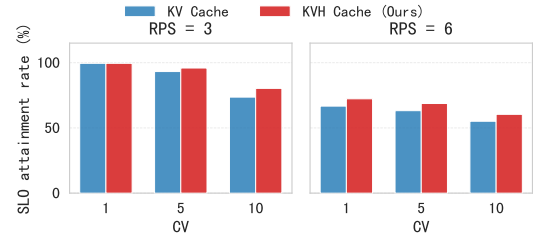


Fig. 13: SLO attainment and throughput comparison without the Adaptive Dynamic Cache mechanism.

capabilities, particularly in tasks demanding rigorous long-context comprehension. Other approaches [27]–[30] investigate KV cache quantization to reduce precision. In contrast to these lossy compression paradigms, EfficientServe introduces a lossless H Cache mechanism. By temporarily offloading standard KV caches into highly compressed intermediate hidden states and leveraging partial recomputation, EfficientServe guarantees exact generation quality while providing critical memory elasticity during traffic bursts.

B. LLM Inference Serving Systems

The rapid proliferation of Large Language Models has catalyzed extensive research into optimizing inference serving systems. Orca [7] pioneered iteration-level scheduling (also known as continuous batching) to significantly improve system throughput compared to request-level batching. To address the severe memory fragmentation caused by dynamic sequence generation, vLLM [15] introduced PagedAttention, which manages the KV cache at the memory block level,

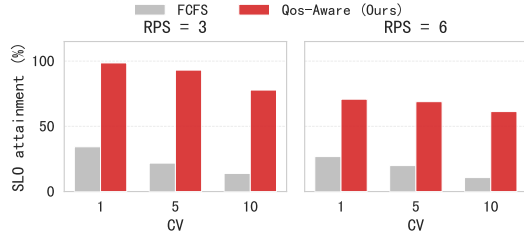


Fig. 14: Impact of the QoS-aware scheduling strategy on TTFT violations and overall system throughput.

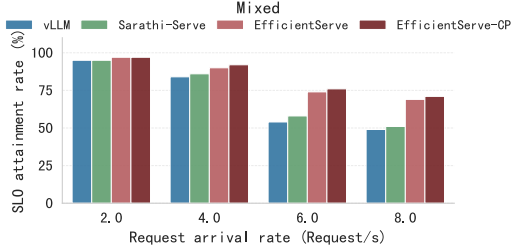


Fig. 15: System throughput comparison after integrating Chunked-Prefill techniques.

mirroring virtual memory paging in operating systems. More recently, systems such as Sarathi [16] and DeepSpeed-FastGen [17] proposed chunked-prefill techniques to split excessively long prefill phases, effectively mitigating GPU compute stalls. However, these systems primarily operate on FCFS principles and static memory allocation, which inherently struggle with the Head-of-Line blocking and memory walls prevalent in highly heterogeneous mixed workloads. Recent new works such as DistServe [31], Splitwise [32], and ShuffleInfer [33] have proposed prefill-decode disaggregated inference serving architectures, which deploy the two phases of LLM inference in isolated or even heterogeneous computing environments. This design eliminates inter-phase resource interference and enables targeted resource scheduling and allocation according to the computational characteristics of each phase. By placing the prefill and decode stages on physically isolated computing resources or dedicated instances, these approaches allow deep, phase-specific optimizations. The research about prefill-decode disaggregated inference is orthogonal to the EfficientServe system proposed in this paper and can be complementary.

C. Scheduling Strategies for LLM Inference

Efficient scheduling is the cornerstone of guaranteeing Quality of Service in multi-scenario workloads. While most mainstream engines default to FCFS policies, recent studies have explored more advanced scheduling paradigms. FastServe [34], for instance, introduced a preemptive multi-level feedback queue utilizing skip-join execution to approximate the Shortest Remaining Processing Time policy. To further address multi-tenant isolation, VTC [35] implements a fair scheduler based on a virtual token counter, ensuring max-min fairness at the token granularity while maintaining high

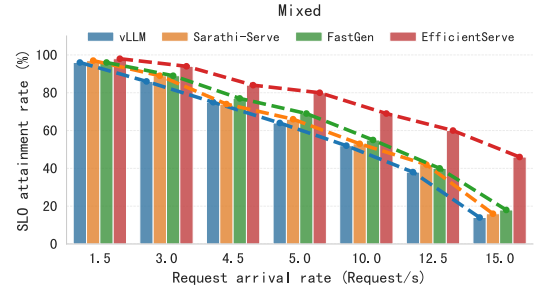


Fig. 16: Performance scaling under distributed Tensor Parallelism (TP=2) deployment.

resource utilization. Nevertheless, without accurate foresight of the generation length, such systems often make suboptimal preemption decisions, leading to unnecessary overhead. EfficientServe differentiates itself by co-designing a lightweight, embedding-based iterative length predictor with a marginal-gain QoS scheduler, enabling proactive and accurate task interleaving to balance TTFT and TTLT.

VI. CONCLUSION

In this paper, we presented EfficientServe, an innovative LLM inference serving system designed for the complexities of real-world mixed workloads. By co-designing an embedding-based length predictor, an adaptive H/KV Cache allocation pool, and a QoS-aware scheduler, EfficientServe effectively neutralizes memory bottlenecks and HoL blocking. Extensive evaluations demonstrate that EfficientServe provides a substantial leap in system throughput and strict SLO adherence, offering a highly practical solution for industrial-scale LLM deployments.

VII. ACKNOWLEDGMENTS

This work was supported by the National Key R&D Program of China (Grant No. 2023YFB3308402)

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] A. Grattafiori, A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Vaughan *et al.*, “The llama 3 herd of models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [3] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang *et al.*, “Qwen technical report,” *arXiv preprint arXiv:2309.16609*, 2023.
- [4] DeepSeek-AI, A. Liu, B. Feng, B. Xue, B. Wang, B. Wu, C. Lu, C. Zhao, C. Deng, C. Zhang, C. Ruan, D. Dai, D. Guo, D. Yang, D. Chen, D. Ji, E. Li, F. Lin, F. Dai, F. Luo, G. Hao, G. Chen, G. Li, H. Zhang, H. Bao, H. Xu, H. Wang, H. Zhang, H. Ding, H. Xin, H. Gao, H. Li, H. Qu, J. L. Cai, J. Liang, J. Guo, J. Ni, J. Li, J. Wang, J. Chen, J. Chen, J. Yuan, J. Qiu, J. Li, J. Song, K. Dong, K. Hu, K. Gao, K. Guan, K. Huang, K. Yu, L. Wang, L. Zhang, L. Xu, L. Xia, L. Zhao, L. Wang, L. Zhang, M. Li, M. Wang, M. Zhang, M. Zhang, M. Tang, M. Li, N. Tian, P. Huang, P. Wang, P. Zhang, Q. Wang, Q. Zhu, Q. Chen, Q. Du, R. J. Chen, R. L. Jin, R. Ge, R. Zhang, R. Pan, R. Wang, R. Xu, R. Zhang, R. Chen, S. S. Li, S. Lu, S. Zhou, S. Chen, S. Wu, S. Ye, S. Ye, S. Ma, S. Wang, S. Zhou, S. Yu, S. Zhou, S. Pan, T. Wang, T. Yun, T. Pei, T. Sun, W. L. Xiao, W. Zeng, W. Zhao, W. An, W. Liu, W. Liang, W. Gao, W. Yu, W. Zhang, X. Q. Li, X. Jin,

- X. Wang, X. Bi, X. Liu, X. Wang, X. Shen, X. Chen, X. Zhang, X. Chen, X. Nie, X. Sun, X. Wang, X. Cheng, X. Liu, X. Xie, X. Liu, X. Yu, X. Song, X. Shan, X. Zhou, X. Yang, X. Li, X. Su, X. Lin, Y. K. Li, Y. Q. Wang, Y. X. Wei, Y. X. Zhu, Y. Zhang, Y. Xu, Y. Xu, Y. Huang, Y. Li, Y. Zhao, Y. Sun, Y. Li, Y. Wang, Y. Yu, Y. Zheng, Y. Zhang, Y. Shi, Y. Xiong, Y. He, Y. Tang, Y. Piao, Y. Wang, Y. Tan, Y. Ma, Y. Liu, Y. Guo, Y. Wu, Y. Ou, Y. Zhu, Y. Wang, Y. Gong, Y. Zou, Y. He, Y. Zha, Y. Xiong, Y. Ma, Y. Yan, Y. Luo, Y. You, Y. Liu, Y. Zhou, Z. F. Wu, Z. Z. Ren, Z. Ren, Z. Sha, Z. Fu, Z. Xu, Z. Huang, Z. Zhang, Z. Xie, Z. Zhang, Z. Hao, Z. Gou, Z. Ma, Z. Yan, Z. Shao, Z. Xu, Z. Wu, Z. Zhang, Z. Li, Z. Gu, Z. Zhu, Z. Liu, Z. Li, Z. Xie, Z. Song, Z. Gao, and Z. Pan, "Deepseek-v3 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2412.19437>
- [5] G. Comanici, E. Bieber, M. Schaeckermann, I. Pasupat, N. Sachdeva, I. Dhillon, M. Blistein, O. Ram, D. Zhang, E. Rosen *et al.*, "Gemini 2.5: Pushing the frontier with advanced reasoning, multimodality, long context, and next generation agentic capabilities," *arXiv preprint arXiv:2507.06261*, 2025.
- [6] K. Team, Y. Bai, Y. Bao, G. Chen, J. Chen, N. Chen, R. Chen, Y. Chen, Y. Chen, Y. Chen *et al.*, "Kimi k2: Open agentic intelligence," *arXiv preprint arXiv:2507.20534*, 2025.
- [7] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for {Transformer-Based} generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.
- [8] Y. Jin, C.-F. Wu, D. Brooks, and G.-Y. Wei, "S3: Increasing gpu utilization during generative inference for higher throughput," *Advances in Neural Information Processing Systems*, vol. 36, pp. 18015–18027, 2023.
- [9] Z. Zheng, X. Ren, F. Xue, Y. Luo, X. Jiang, and Y. You, "Response length perception and sequence scheduling: An llm-empowered llm inference pipeline," *Advances in Neural Information Processing Systems*, vol. 36, pp. 65517–65530, 2023.
- [10] S. Choi, J. Goo, E. Jeon, M. Yang, and M. Jang, "Elis: Efficient llm iterative scheduling system with response length predictor," *arXiv preprint arXiv:2505.09142*, 2025.
- [11] H. Qiu, W. Mao, A. Patke, S. Cui, S. Jha, C. Wang, H. Franke, Z. T. Kalbarczyk, T. Başar, and R. K. Iyer, "Efficient interactive llm serving with proxy model-based sequence length prediction," *arXiv preprint arXiv:2404.08509*, 2024.
- [12] NVIDIA, "NVIDIA NVLink and NVSwitch: The world's most advanced high-speed interconnect," <https://www.nvidia.com/en-sg/data-center/nvlink/>, 2026, accessed: 2026-02-05. [Online]. Available: <https://www.nvidia.com/en-sg/data-center/nvlink/>
- [13] J. Bai, S. Bai, Y. Chu, Z. Cui, K. Dang, X. Deng, Y. Fan, W. Ge, Y. Han, F. Huang, B. Hui, L. Ji, M. Li, J. Lin, R. Lin, D. Liu, G. Liu, C. Lu, K. Lu, J. Ma, R. Men, X. Ren, C. Ren, C. Tan, S. Tan, J. Tu, P. Wang, S. Wang, W. Wang, S. Wu, B. Xu, J. Xu, A. Yang, H. Yang, J. Yang, S. Yang, Y. Yao, B. Yu, H. Yuan, Z. Yuan, J. Zhang, X. Zhang, Y. Zhang, Z. Zhang, C. Zhou, J. Zhou, X. Zhou, and T. Zhu, "Qwen technical report," 2023. [Online]. Available: <https://arxiv.org/abs/2309.16609>
- [14] Qwen, :, A. Yang, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Li, D. Liu, F. Huang, H. Wei, H. Lin, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Lin, K. Dang, K. Lu, K. Bao, K. Yang, L. Yu, M. Li, M. Xue, P. Zhang, Q. Zhu, R. Men, R. Lin, T. Li, T. Tang, T. Xia, X. Ren, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Wan, Y. Liu, Z. Cui, Z. Zhang, and Z. Qiu, "Qwen2.5 technical report," 2025. [Online]. Available: <https://arxiv.org/abs/2412.15115>
- [15] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles*, ser. SOSP '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 611–626. [Online]. Available: <https://doi.org/10.1145/3600006.3613165>
- [16] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," in *Proceedings of the 18th USENIX Conference on Operating Systems Design and Implementation*, ser. OSDI'24. USA: USENIX Association, 2024.
- [17] C. Holmes, M. Tanaka, M. Wyatt, A. A. Awan, J. Rasley, S. Rajbhandari, R. Y. Aminabadi, H. Qin, A. Bakhtiari, L. Kurilenko, and Y. He, "DeepSpeed-fastgen: High-throughput text generation for llms via mii and deepspeed-inference," 2024. [Online]. Available: <https://arxiv.org/abs/2401.08671>
- [18] S. Team, "Sharegpt: Share your wildest conversations with one click," <https://sharegpt.com>, 2023, deprecated. Please use OpenAI's built-in sharing instead.
- [19] Taatsu-lab, "Alpaca: A strong, replicable instruction-following model," <https://huggingface.co/datasets/taatsu-lab/alpaca>, 2023, accessed: [Insert Date].
- [20] Ajibawa, "Python-code-23k-sharegpt: A dataset of 23k python code examples from sharegpt," 2023. [Online]. Available: <https://huggingface.co/datasets/ajibawa-2023/Python-Code-23k-ShareGPT>
- [21] CCDV, "arxiv-summarization: A dataset of arxiv paper summaries," 2023. [Online]. Available: <https://huggingface.co/datasets/ccdv/arxiv-summarization>
- [22] OpenAI, "Gsm8k: Grade school math 8k dataset," 2021. [Online]. Available: <https://huggingface.co/datasets/openai/gsm8k>
- [23] Y. Wang, Y. Chen, Z. Li, X. Kang, Y. Fang, Y. Zhou, Y. Zheng, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, and X. Chu, "Burstgpt: A real-world workload dataset to optimize llm serving systems," in *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining V2*, ser. KDD '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 5831–5841. [Online]. Available: <https://doi.org/10.1145/3711896.3737413>
- [24] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett *et al.*, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 34661–34710, 2023.
- [25] Z. Liu, A. Desai, F. Liao, W. Wang, V. Xie, Z. Xu, A. Kyrillidis, and A. Shrivastava, "Scissorhands: Exploiting the persistence of importance hypothesis for llm kv cache compression at test time," in *Advances in Neural Information Processing Systems*, A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine, Eds., vol. 36. Curran Associates, Inc., 2023, pp. 52342–52364.
- [26] W. Lee, J. Lee, J. Seo, and J. Sim, "[InfiniGen]: Efficient generative inference of large language models with dynamic {KV} cache management," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 155–172.
- [27] C. Hooper, S. Kim, H. Mohammadzadeh, M. W. Mahoney, Y. S. Shao, K. Keutzer, and A. Gholami, "Kvquant: Towards 10 million context length llm inference with kv cache quantization," *Advances in Neural Information Processing Systems*, vol. 37, pp. 1270–1303, 2024.
- [28] S. Ashkboos, A. Mohtashami, M. L. Croci, B. Li, P. Cameron, M. Jaggi, D. Alistarh, T. Hoefler, and J. Hensman, "Quarot: Outlier-free 4-bit inference in rotated llms," *Advances in Neural Information Processing Systems*, vol. 37, pp. 100213–100240, 2024.
- [29] Y. Zhao, C.-Y. Lin, K. Zhu, Z. Ye, L. Chen, S. Zheng, L. Ceze, A. Krishnamurthy, T. Chen, and B. Kasicki, "Atom: Low-bit quantization for efficient and accurate llm serving," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 196–209, 2024.
- [30] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International conference on machine learning*. PMLR, 2023, pp. 38087–38099.
- [31] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.
- [32] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 118–132.
- [33] C. Hu, H. Huang, L. Xu, X. Chen, C. Wang, J. Xu, S. Chen, H. Feng, S. Wang, Y. Bao *et al.*, "ShuffleInfer: Disaggregate llm inference for mixed downstream workloads," *ACM Transactions on Architecture and Code Optimization*, 2025.
- [34] B. Wu, Y. Zhong, Z. Zhang, S. Liu, F. Liu, Y. Sun, G. Huang, X. Liu, and X. Jin, "Fast distributed inference serving for large language models," *arXiv preprint arXiv:2305.05920*, 2023.
- [35] Y. Sheng, S. Cao, D. Li, B. Zhu, Z. Li, D. Zhuo, J. E. Gonzalez, and I. Stoica, "Fairness in serving large language models," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 965–988.